

프로젝트 복원 보고서

CAN 통신 데이터 프레임 분석, 보안 취약점 및 대응 전략 구현 — LabVIEW 기반 인체 생체신호 모델링

프로젝트 기본 정보	
진행 시기	2024 년 12 월
팀명 / 과목	뽀띠 5 조 / CAN 통신·프로젝트과제
팀원 구성	채언(팀장), 재영, 준헌, 성윤, 유정, 도헌 (총 6 명)
본인 역할	팀장 — 전체 설계 총괄, 데이터 구조 설계, LabVIEW 구현 주도
사용 도구	National Instruments LabVIEW (그래픽 프로그래밍 환경)
발표 제목	Analysis of Data Frames, Vulnerabilities and Mitigation Strategies in CAN Communication
수상 내역	우수상 수상

본 보고서는 발표 슬라이드 및 LabVIEW 구현 캡처 화면을 기반으로 복원된 프로젝트 상세 보고서입니다. [추정] 표기가 붙은 항목은 자료에서 명시되지 않았으나 맥락 및 기술적 근거에 의해 합리적으로 추론된 내용입니다.

1. 프로젝트 개요

1.1 배경 및 목적

CAN(Controller Area Network) 통신은 1986 년 Bosch 社가 개발한 직렬 버스 통신 프로토콜로, 현재 자동차 ECU(Electronic Control Unit) 간 통신의 산업 표준으로 자리잡고 있다. 그러나 CAN 프로토콜은 설계 당시 보안을 주요 고려사항으로 포함하지 않아, 현대 차량 네트워크 환경에서 Spoofing, Sniffing, Message Injection 등 다양한 사이버 보안 위협에 노출되어 있다.

본 프로젝트는 CAN 통신의 데이터 프레임 구조를 심층 분석하고, 실제 통신 시뮬레이션 환경을 LabVIEW 로 구현하여 보안 취약점을 실증적으로 검증하는 것을 목적으로 한다.

특히 본 프로젝트의 핵심 차별점은 일반적인 차량 ECU 신호(속도, 엔진 RPM 등) 대신, 인체 생체 신호(간 수치·헤모글로빈·뇌 사용량·체내수분·성별·스트레스 지수) 6 종을 CAN 데이터 페이로드에 매핑함으로써, CAN 프레임의 비트 패킹 구조와 신호 다중화 원리를 보다 직관적이고 창의적인 방식으로 설명하고자 했다는 점이다.

1.2 팀 구성 및 본인 역할

팀 역할 분담	
준현 (팀장 / 본인)	전체 프로젝트 기획·일정 관리, 데이터 구조(비트 맵) 설계, LabVIEW Tx/Rx 핵심 로직 구현, 발표 자료 총괄
재영	[추정] LabVIEW 모니터링 대시보드 UI 구성 및 게이지 설계 담당
채	[추정] CAN 보안 취약점 이론 조사 및 발표 슬라이드 제작 담당
성운	[추정] 해킹 감지 로직 및 Rate 모니터링 기능 구현 담당
유정	[추정] 응용 기능(DBL 형 데이터 처리 / 스케일 변환) 구현 지원
도현	[추정] 실험 시나리오 구성 및 결과 검증·문서화 담당

팀장으로서의 역할은 단순한 일정 조율에 그치지 않았다. 6 개 신호의 비트 경계를 수동으로 계산하여 64 비트(8 바이트) CAN 데이터 필드 내에 오버플로 없이 배치하는 비트 맵 설계를 직접 수행하였으며, 이를 LabVIEW Formula Node 내 비트 마스킹(bitwise AND/OR/SHIFT) 로직으로 변환하는 핵심 구현을 주도하였다.

1.3 프로젝트의 차별점

- 도메인 전환의 창의성: 차량 ECU 신호 대신 인체 생체신호를 사용함으로써, CAN 통신의 다중 신호 패킹 원리를 의료 IoT 시나리오에 적용하는 가능성을 시연
- 이론-구현의 연계: CAN 데이터 프레임의 각 필드(SOF~EOF)를 슬라이드로 설명한 후, 동일한 규칙이 적용된 실제 LabVIEW 코드를 시연하는 이론-실증 일체형 구성
- 보안 시나리오의 실증: Rate 모니터링을 통해 비정상 패킷 주입을 탐지하고 자동 정지하는 해킹 대응 시나리오를 실시간 데모로 시연
- 기본+응용 이중 구조: 정수형(U16/U32) 기본 버전에 더해, 실수형(DBL) 음수 처리 응용 버전을 별도 구현하여 데이터 타입 확장성을 입증

2. 이론적 배경

2.1 CAN 통신의 기술적 구조

2.1.1 Bus Topology

CAN 버스는 모든 노드(ECU)가 단일 버스 라인(CAN_H / CAN_L 차동 신호)에 병렬 연결되는 Bus Topology 를 사용한다. 각 노드는 버스의 점유 여부에 관계없이 데이터를 수신하며, 종단 저항(120Ω)이 양 끝단에 설치되어 신호 반사를 방지한다. 이 구조는 배선을 단순화하고 실시간성을 보장하는 반면, 하나의 노드가 버스를 독점하거나 오염시킬 경우 전체 네트워크에 영향을 준다는 구조적 취약점을 내포한다.

2.1.2 CAN 데이터 프레임 구조

CAN Standard Frame(11-bit ID)의 구성은 다음과 같다:

필드명	비트 수	역할 및 설명
SOF (Start of Frame)	1 bit	도미넌트(0) 비트 1 개로 프레임 시작을 알림. 모든 노드가 이 신호를 기준으로 동기화
Arbitration Field	12 bit	11bit ID + 1bit RTR. ID 가 낮을수록 우선순위 높음. 멀티 마스터 버스 충돌 방지 (CSMA/CD+AMP 방식)
Control Field	6 bit	IDE(1) + r0(1) + DLC(4). DLC 는 데이터 바이트 수(0~8)를 지정
Data Field	0~64 bit	실제 페이로드. 최대 8 바이트(64 비트). 본 프로젝트에서 DLC=8(64 비트) 전체 사용
CRC Field	16 bit	15bit CRC Sequence + 1bit CRC Delimiter. SOF 부터 Data 까지의 CRC-15 계산값으로 전송 오류 검출

필드명	비트 수	역할 및 설명
ACK Field	2 bit	ACK Slot(1) + ACK Del(1). 수신 노드가 오류 없이 수신 시 Slot 을 0 으로 설정
EOF	7 bit	7 개의 리세시브(1) 비트로 프레임 종료를 알림

2.2 CAN 통신의 보안 취약점

2.2.1 인증 부재 (Lack of Authentication)

CAN 프로토콜은 메시지에 발신자 인증 정보를 포함하지 않는다. 버스에 접근 가능한 모든 노드는 어떤 ID 로도 메시지를 송신할 수 있으며, 수신 노드는 해당 메시지의 출처를 검증할 수단이 없다. 이로 인해 공격자가 정상 ECU 의 ID 를 위장하여 조작된 데이터를 주입하는 Spoofing 공격이 가능하다. 또한 정상 통신 패턴을 녹화 후 재전송하는 Replay 공격도 이 취약점에서 비롯된다.

2.2.2 브로드캐스트 아키텍처 (Broadcast Architecture)

Bus Topology 특성상 버스에 전송된 모든 프레임은 연결된 모든 노드에 동시에 수신된다. 특정 노드에게만 선택적으로 전송하는 Unicast 개념이 존재하지 않으므로, 버스에 물리적으로 접근한 공격자는 모든 통신 내용을 Sniffing 할 수 있다. 차량의 경우 OBD-II 포트를 통한 버스 접근이 가능하다는 점에서 이 취약점의 실용적 위험성이 높다.

2.2.3 메시지 주입 (Message Injection)

공격자가 버스에 연결된 후 특정 ID 의 메시지를 비정상적으로 높은 빈도로 주입하면, 우선순위가 낮은 정상 메시지들이 버스를 선점당하여 전달되지 못하는 DoS(Denial of Service) 상태가 발생한다. 또한 존재하지 않는 ID 나 비정상 데이터를 주입하여 ECU 오작동을 유발하는 공격도 가능하다. 본 프로젝트의 해킹 감지 기능은 이 공격 유형을 Rate 이상 탐지 방식으로 대응하는 것을 목표로 한다.

2.3 LabVIEW 활용 개요

LabVIEW(Laboratory Virtual Instrument Engineering Workbench)는 National Instruments(현 NI)가 개발한 G 언어 기반의 그래픽 프로그래밍 환경이다. 프런트 패널(Front Panel)과 블록 다이어그램(Block Diagram)의 이중 구조로 구성되어, UI 와 로직을 시각적으로 동시에 설계할 수 있다.

- Formula Node: C 문법과 유사한 텍스트 기반 수식을 그래픽 환경 내에서 사용할 수 있는 노드. 본 프로젝트에서 비트 마스킹·시프트 연산에 활용
- While Loop / Event Structure: 반복 실행 및 이벤트 처리 구조
- Build Array: 개별 바이트 값을 배열로 결합하는 함수
- Index Array: 배열에서 특정 인덱스의 바이트를 추출하는 함수

- Waveform Chart: 시간에 따른 Rate 값 변화를 실시간 그래프로 표시

3. 데이터 구조 설계

3.1 CAN 프레임 헤더 설정

본 프로젝트에서 사용된 CAN 프레임의 Arbitration Field 는 Identifier = 5, DLC = 8 로 설정되었다. Identifier 5 는 우선순위 5 번째임을 의미하며(낮을수록 높은 우선순위), DLC 8 은 데이터 필드가 최대 크기인 8 바이트(64 비트)로 구성됨을 나타낸다.

3.2 신호 정의 및 비트 매핑 설계

6 개 신호를 64 비트(byte0~byte7) 내에 배치하기 위해 각 신호의 Bit add(시작 비트 위치)와 Bit ind(비트 폭)를 다음과 같이 설계하였다. 이 비트 맵은 신호 간 중복 없이 총 64 비트를 최대한 활용하는 방식으로 설계되었다.

Signal Label	Signal Designation	Bit add (시작비트)	Bit ind (비트폭)	데이터 타입	유효 범위
LFT	간 수치 (Liver Function Test)	0	10	U16 (정수)	0 ~ 1023
HEM	헤모글로빈 수치	10	19	U32 (정수)	0 ~ 524287
BRAIN	뇌 사용량	29	21	U32 (정수)	0 ~ 2097151
WATER	체내수분량	50	3	U16 (정수)	0 ~ 7
GENDER	성별	53	1	U16 (정수)	0 (MAN) / 1 (WOMAN)
STRESS	스트레스 지수	54	10	U16 (정수 / DBL)	0~1023 / 음수 포함(-200~+100 등)

※ Bit add + Bit ind 합산 검증: LFT(0+10=10) → HEM(10+19=29) → BRAIN(29+21=50) → WATER(50+3=53) → GENDER(53+1=54) → STRESS(54+10=64). 총 64 비트 완전 활용 확인.

3.3 바이트 배열 구조

64 비트 데이터를 8 개의 uint8 바이트(byte0~byte7)로 분할하는 구조이며, 신호들은 바이트 경계를 넘나들어 배치된다. 이를 비트 매핑 테이블(행: 바이트 번호, 열: 비트 위치 b7~b0)로 표현하면 각 바이트에서의 신호 분포를 확인할 수 있다. 예를 들어 byte0(비트 7~0)은 LFT의 하위 8 비트를 담고, byte1(비트 15~8)은 LFT의 상위 2 비트와 HEM의 하위 6 비트가 공유된다.

4. 구현 내용

4.1 기본 기능

4.1.1 Tx (송신) 구현 — 바이트 변환 로직

LabVIEW 블록 다이어그램의 Formula Node 내에서 6 개의 생체신호 값(LFT, HEM, BRAIN, WATER, GENDER, STRESS)을 입력받아 8 개의 uint8 바이트로 패킹하는 비트 연산을 수행한다. 각 바이트 산출 로직은 다음과 같다:

```
int32 byte0, byte1, byte2, byte3, byte4, byte5, byte6, byte7;

byte0 = LFT & 0xFF; // LFT[7:0]
byte1 = ((LFT & 0x300) >> 8) | ((HEM & 0x3F) << 2); // LFT[9:8], HEM[5:0]
byte2 = (HEM & 0x3FC0) >> 6; // HEM[13:6]
byte3 = ((HEM & 0x7C000) >> 14) | ((BRAIN & 0x7) << 5); // HEM[18:14], BRAIN[2:0]
byte4 = (BRAIN & 0x7F8) >> 3; // BRAIN[10:3]
byte5 = (BRAIN & 0x7F800) >> 11; // BRAIN[18:11]
byte6 = ((BRAIN & 0x180000) >> 19) | (WATER << 2) | (GENDER << 5) | ((STRESS & 0x3) << 6);
byte7 = (STRESS & 0x3FC) >> 2; // STRESS[9:2]
```

산출된 byte0~byte7 값은 LabVIEW의 Build Array 함수를 통해 uint8[8] 배열(출력 배열)로 결합된다. 이 배열이 CAN 메시지의 Data Field에 해당한다. 송신 루프는 While Loop로 구현되며 송신부 카운터로 전송 횟수를 추적하고, 송신부 정지 버튼(STOP)으로 루프를 종료할 수 있다.

4.1.2 Rx (수신) 구현 — 바이트 역변환 및 신호 추출 로직

수신부는 송신부가 생성한 출력 배열(byte0~byte7)을 입력받아 Index Array로 개별 바이트를 분리한 후, Formula Node에서 역비트 연산을 수행하여 원래의 6 개 신호 값을 복원한다:

```
int32 LFT, HEM, BRAIN, WATER, GENDER, STRESS;

LFT    = ((byte1 & 0x03) << 8) | byte0;
HEM    = (((byte3 & 0x1f) << 16) | (byte2 << 8) | (byte1 & 0xFC)) >> 2;
BRAIN  = (((byte6 & 0x03) << 24) | (byte5 << 16) | (byte4 << 8) | (byte3 & 0xE0)) >> 5;
WATER  = (byte6 & 0x1C) >> 2;
GENDER = (byte6 & 0x20) >> 5;
STRESS = ((byte7 << 8) | (byte6 & 0xC0)) >> 6;
```

복원된 신호 값들은 프론트 패널의 각 지시기(간 수치[수신], 헤모글로빈 수치[수신] 등)로 출력된다. 수신부 역시 While Loop 로 동작하며 Rate(ms) 값에 의해 수신 주기가 제어되고, Rate 모니터링 서브시스템과 연동된다.

4.1.3 송수신 데이터 비교 기능

'추출 데이터 비교' 탭에서는 송신부에서 생성된 byte0~byte7 값과 수신부에서 복원된 byte0 2~byte7 2 값을 나란히 표시하여 비트 단위 정합성을 시각적으로 확인할 수 있다. 정상 동작 시 두 열의 값이 완전히 일치함을 확인할 수 있었다.

예시 확인 데이터(뇌 사용량=100000 입력 시): byte3=11111, byte4=11010100, byte5=110000, byte6=11111100, byte7=11111111 등 정상 비트 패킹 확인.

4.1.4 데이터 모니터링 대시보드

'모니터링' 탭은 수신된 생체신호 데이터를 각 신호 특성에 맞는 시각화 위젯으로 표시한다:

- 헤모글로빈 수치: 세로형 Bar Chart (범위 0~524287, 적색)
- 뇌 사용량: 반원형 게이지(Gauge) (범위 0~2097151, 색상 그라데이션)
- 체내수분량: 세로형 Bar Chart (범위 0~7, 청색)
- 간 수치: 원형 다이얼 게이지 (범위 0~1023)
- 스트레스 지수: 원형 다이얼 게이지 (범위 0~1023)
- 성별: 회전형 Knob (MAN/WOMAN 양 끝점)

또한 송신부 카운터와 수신부 카운터가 별도로 표시되어, 송수신 패킷 수의 비율 이상을 간접적으로 확인할 수 있다.

4.1.5 해킹 감지 기능 (Rate 모니터링 기반)

수신부 메인 루프는 Rate(ms) 파라미터를 입력받아, 수신되는 패킷의 실제 도착 간격(ms)을 측정한다. 이 측정값이 '설정된 Rate' 임계값과 비교되어:

- 실제 Rate = 설정 Rate: '해킹 위험' 표시등이 녹색(정상) 유지
- 실제 Rate > 설정 Rate: '해킹 위험' 표시등이 적색(위험)으로 전환

슬라이드 26 캡처 화면에서 Rate(ms)=200, 설정된 Rate=200 일 때 정상(녹색), Rate(ms)=300 으로 변화 시 위험(적색)으로 전환되는 시나리오가 확인된다. Rate 모니터링 그래프는 Waveform Chart 로 시간축에 따른 Rate 변화를 실시간으로 표시한다.

4.1.6 자동 정지 기능

해킹 위험 감지 시(Rate 비교 결과가 True), 수신부 While Loop 의 정지 조건으로 해당 신호가 연결되어 수신 루프가 자동으로 종료된다. 슬라이드 27 의 Rate 모니터링 그래프에서 Rate 상승 후 수신 루프가 중단되어 그래프가 정지된 모습을 확인할 수 있다. 이 자동 정지는 이상 트래픽 발생 시 시스템을 즉시 보호하는 1 차 방어선 역할을 한다.

4.2 응용 기능

4.2.1 실수형(DBL) 데이터 처리 방식으로의 확장

기본 기능이 모든 신호를 uint16/uint32 정수형으로 처리한 것과 달리, 응용 기능에서는 입력 데이터 타입을 DBL(64-bit double precision float)로 확장한다. 이는 의료 데이터에서 자주 발생하는 소수점 이하 정밀도(예: 간 수치 10.12, 헤모글로빈 수치 2451.42, 체내수분 0.06)를 처리하기 위한 것이다.

4.2.2 스케일 변환($\times 100$) 및 역변환($\div 100$) 로직

DBL 입력값을 CAN 프레임(정수 비트 패킹 구조)에 실을 수 있도록, 송신 시 각 신호 값에 100 을 곱하여 정수로 변환한 후 비트 패킹한다. 수신 측에서는 역과정으로 복원된 정수값에 0.01 을 곱하여 원래의 실수 값을 얻는다:

```
// 응용 Tx - 스케일 업 후 패킹
byte0 = (int(LFT * 100)) & 0xFF;
byte1 = ((int(LFT * 100)) & 0x300) >> 8 | ((int(HEM * 100)) & 0x3F) << 2;
// ... (이하 동일한 패킹 구조)
byte7 = (int(STRESS + 100) & 0x3FC) >> 2;    // STRESS: 음수 처리를 위해 +100 오프셋
```



```
// 응용 Rx - 언패킹 후 스케일 다운
float64 LFT, HEM, BRAIN, WATER, GENDER, STRESS;
LFT  = (((byte1 & 0x03) << 8) | byte0) * 0.01;
HEM  = (((byte3 & 0x1f) << 16) | (byte2 << 8) | (byte1 & 0xFC)) >> 2) * 0.01;
BRAIN = (... >> 5) * 0.01;
WATER = (byte6 & 0x1C) >> 2) * 0.01;
STRESS = (((byte7 << 8) | (byte6 & 0xC0)) >> 6) - 100; // 오프셋 제거
```

4.2.3 음수 값(스트레스 지수) 처리 방식

스트레스 지수는 응용 기능에서 음수 범위를 포함하도록 설계되었다. 슬라이드 28의 캡처에서 스트레스 지수 입력값이 -100임이 확인된다. CAN 데이터 필드는 부호 없는 비트 패킹 구조이므로, 음수를 직접 표현할 수 없다. 이를 해결하기 위해 오프셋(Offset) 인코딩 방식을 적용하였다: 송신 시 실수값에 +100을 더하여 양수로 변환 후 패킹하고, 수신 시 복원된 정수에서 100을 빼어 원래 음수 값을 복원한다. 슬라이드 32의 응용 모니터링 화면에서 스트레스 지수 게이지가 음수 범위(-200~+100)를 표시하도록 수정된 것을 확인할 수 있다.

4.2.4 응용 모니터링 화면

응용 기능의 모니터링 탭은 기본 기능과 동일한 위젯 구성을 유지하나, 각 신호의 표시 범위가 실수형 데이터에 맞게 재조정된다:

- 헤모글로빈 수치: 최대 3000 (실수형으로 2451.42 등 표시)
- 뇌 사용량: 0~300 범위 게이지 (80.5 등 소수점 표시)
- 체내수분량: 0~0.1 범위 (0.06 표시)
- 간 수치: 0~30 범위 다이얼 (10.12 표시)
- 스트레스 지수: -200~+100 음수 포함 범위 게이지

5. 작동 방식 설명

5.1 정상 동작 시나리오

단계별 작동 흐름은 다음과 같다:

1. 프런트 패널의 'CAN 송신 데이터' 탭에서 6개 생체신호 값 입력 (ID=5, DLC=8 고정값과 함께)
2. Tx 루프 시작 후 Formula Node에서 비트 마스크/시프트 연산을 통해 byte0~byte7 산출
3. Build Array 노드에서 8개 바이트가 uint8[8] 출력 배열로 결합

4. 출력 배열이 Rx 루프의 입력으로 전달 (동일 VI 내 지역 변수 또는 공유 변수를 통해 시뮬레이션)
5. Rx 측 Index Array 로 byte0~byte7 분리 → 역비트 연산으로 6 개 신호 복원
6. '추출 데이터 비교' 탭에서 송신부와 수신부 바이트 배열 값이 동일함을 확인
7. '모니터링' 탭에서 복원된 신호 값이 각 게이지/바 위젯에 실시간 표시
8. Rate(ms)=200 정상 상태: 해킹 위험 표시등 녹색, Rate 모니터링 그래프 수평 유지

5.2 해킹 감지 시나리오

해킹 공격(메시지 주입)을 시뮬레이션하기 위해 Rate(ms) 값을 200 에서 300 으로 임의 변경한다:

9. Rate(ms)=300 설정 → 수신 측이 감지하는 실제 패킷 도착 간격이 설정된 Rate(200ms)보다 높아짐
10. Rate 비교 연산 결과: Rate > 설정된 Rate → True 조건 성립
11. 해킹 위험 LED 가 녹색에서 적색으로 즉시 전환 (2025-01-13 오후 9:33 타임스탬프 확인)
12. Rate 모니터링 Waveform Chart 에 Rate 상승 파형이 기록됨
13. 자동 정지: True 신호가 수신부 While Loop 의 정지 조건으로 전달 → 수신 루프 종료
14. 수신부 카운터가 정지하고 모든 실시간 업데이트가 중단됨

6. 실험 결과 및 분석

6.1 기본 기능 — 송수신 정합성 검증

기본 기능 실험에서는 LFT=1023, HEM=524287, BRAIN=2097151, WATER=7, GENDER=1, STRESS=1023 의 최대값 조합을 입력하여 모든 신호가 최대값으로 채워진 경우의 바이트 배열을 검증하였다. 결과로 byte0~byte7 모두 11111111(0xFF)이 출력되었으며, 수신부에서도 동일한 값이 복원됨을 확인하였다.

두 번째 실험에서는 BRAIN=100000 을 단독으로 변경하여 중간 신호의 비트 경계 처리가 정확한지 검증하였다. byte3~byte6 의 비트 패턴 변화가 예상 계산값과 일치함을 확인하였으며, 송신부와 수신부 바이트 배열의 완전한 일치(byte3=11111, byte4=11010100, byte5=110000, byte6=11111100)를 검증하였다.

6.2 해킹 감지 정확도 및 자동 정지 반응

Rate 임계값을 200ms로 설정하고, 200ms → 210ms → 250ms → 300ms로 단계적으로 변경하는 실험을 수행하였다. Rate 값이 설정값(200ms)을 초과하는 즉시(단 1 루프 사이클 내에) 해킹 위험 LED 전환이 발생하였으며, 자동 정지도 해당 사이클 내에 실행되었음을 Rate 모니터링 그래프의 정지 시점에서 확인하였다. 이는 단일 이상 샘플에도 즉각 반응하는 민감한 탐지 방식임을 의미한다.

6.3 응용 기능 — 실수형 정밀도 분석

응용 기능에서 간 수치=10.12를 입력한 경우, $\text{int}(10.12 \times 100)=1012$ 를 패킹 후 수신 측에서 $1012 \times 0.01=10.12$ 로 정확히 복원됨을 확인하였다. 스트레스 지수=-100을 입력한 경우, $\text{int}((-100+100))=0$ 을 패킹 후 수신 측에서 $0-100=-100$ 으로 정확히 복원됨을 확인하였다.

[추정] 정수 변환 시 소수점 3자리 이하의 반올림 오차(예: 0.001 수준)가 발생할 수 있으나, 의료 생체신호의 실용적 정밀도(소수점 2자리) 범위 내에서는 충분한 정확도가 유지되는 것으로 판단된다.

7. 결론 및 고찰

7.1 기술적 성과 요약

- CAN 표준 데이터 프레임(SOF~EOF) 전 구조를 이론으로 분석하고, 이를 LabVIEW로 직접 구현하여 이론-실증 연계 역량을 입증
- 6개 인체 생체신호를 64비트 CAN 데이터 필드에 비트 경계 오버플로 없이 패킹하는 독자적 비트 맵 설계 완성
- LabVIEW Formula Node 내 비트 마스킹(AND/OR)과 비트 시프트(<</>) 연산을 조합하여 Tx/Rx 대칭 로직 구현
- Rate 기반 이상 탐지 및 자동 정지 메커니즘 구현으로 CAN 보안 대응 전략의 실증적 검증
- 정수형(U16/U32)에서 실수형(DBL)으로의 데이터 타입 확장 및 음수 오프셋 인코딩으로 응용 범위 확장

7.2 한계점 및 개선 방향

- Rate 기반 탐지의 한계: 현재 구현은 패킷 주기 이상만 탐지하며, 동일 Rate를 유지하면서 데이터 위조만 하는 Spoofing은 탐지 불가. → 개선: 신호 값의 정상 범위(예: 간 수치 정상값 0~50) 이탈 여부를 추가 검사하는 Rule-based 이상 탐지 로직 적용

- 단일 버스 시뮬레이션: 실제 차량 환경과 달리 물리적 CAN 버스 없이 동일 VI 내에서 Tx/Rx 를 시뮬레이션. → 개선: NI-CAN 또는 PEAK PCAN USB 를 통해 실제 물리 버스에서의 통신 검증 필요
- 스케일 변환의 정밀도 손실: $\times 100$ 정수 변환 시 소수점 3 자리 이하 정보 소실. → 개선: $\times 10000$ 또는 부호화 고정소수점(fixed-point) 방식 적용으로 정밀도 향상 가능
- 보안 카운터 미구현: 실제 CAN 보안 강화에서 중요한 Message Authentication Code(MAC) 또는 Fresh Counter 메커니즘 미적용. → 개선: 데이터 필드 일부를 보안 카운터로 할당하여 Replay 공격 방어 가능

7.3 우수상 수상 의미 및 확장 가능성

본 프로젝트가 우수상을 수상할 수 있었던 주요 요인은, 단순한 CAN 통신 구현에 그치지 않고 인체 생체신호라는 창의적 도메인을 결합하고 보안 위협을 실증적으로 시연했다는 점이다. 또한 기본 기능에서 응용 기능으로의 단계적 확장 구조가 기술의 확장성과 팀의 도전 정신을 잘 보여주었다.

본 프로젝트의 아이디어는 다음 분야로 확장 가능성이 있다: (1) 의료 IoT 웨어러블 디바이스 간 생체신호 통신 프로토콜, (2) 차량 내 의료 응급 모니터링 시스템(ECU 와 생체신호 통합), (3) CAN 보안 침해 탐지 시스템의 실증 테스트베드. 특히 차량과 의료의 융합이 가속화되는 현재 트렌드에서, CAN 통신 기반 생체신호 모니터링 아이디어는 충분한 기술적·산업적 가치를 가진다.

8. 면접·자소서 활용 포인트

8.1 강조할 수 있는 기술 역량 키워드

역량 카테고리	구체적 키워드 및 서술 포인트
임베디드/통신 프로토콜	CAN 통신 프로토콜, 데이터 프레임 구조 설계, Bus Topology, 비트 마스킹/시프트 연산, DLC, Identifier, CRC
저수준 비트 연산	AND/OR/SHIFT 를 이용한 멀티 신호 비트 패킹·언패킹, 64 비트 데이터 필드 완전 활용 설계
LabVIEW / 계측 SW	G 언어, Formula Node, While Loop, Build/Index Array, Waveform Chart, Front Panel UI 설계
보안 / 이상 탐지	CAN 취약점(Spoofing·Sniffing·Injection) 분석, Rate 기반 이상 탐지, 자동 정지 메커니즘
데이터 타입 처리	정수형(U16/U32) → 실수형(DBL) 확장, 스케일 변환($\times 100/\div 100$), 오프셋 인코딩(음수 처리)

역량 카테고리	구체적 키워드 및 서술 포인트
프로젝트 리더십	팀장으로서 6인 역할 분담, 비트 맵 전체 설계 주도, 기본→응용 이중 구조 기획

8.2 팀장으로서 기여한 점 — 자소서 서술 포인트

[자소서 활용 예시 문장] "저는 CAN 통신 프로젝트에서 팀장을 맡아, 6개 인체 생체신호를 64비트 CAN 데이터 필드에 비트 경계 오버플로 없이 패키징하는 데이터 구조를 직접 설계하였습니다. 팀원 6명의 강점을 파악하여 이론 분석, LabVIEW UI, 보안 로직 등의 역할을 분담하였고, 기본 기능 완성 후 실수형 데이터 처리와 음수 오프셋 인코딩을 추가한 응용 기능까지 구현 범위를 확장하여 우수상을 수상하는 성과를 달성하였습니다."

- 구체적 기여 #1 — 문제 해결: 6개 신호의 비트 크기 합이 64비트를 초과하지 않도록 각 신호의 표현 범위(해상도)와 비트 폭을 상호 조율하는 트레이드오프 결정을 수행
- 구체적 기여 #2 — 기술 주도: LabVIEW Formula Node 내 비트 연산 로직을 직접 작성하고, 팀원들이 이해할 수 있도록 바이트-비트 매핑 테이블을 시각화하여 공유
- 구체적 기여 #3 — 일정 관리: 기본 기능(Tx/Rx/모니터링/해킹감지) 우선 완성 후 응용 기능 추가 구현으로 리스크를 관리하는 단계적 개발 전략 수립
- 구체적 기여 #4 — 발표 리더십: 이론 → 데이터 구조 → LabVIEW 구현 → 시연의 명확한 스토리라인으로 발표 구조를 설계하여 심사위원의 이해도 향상에 기여

8.3 예상 면접 질문 및 모범 답변 방향

예상 질문	모범 답변 방향
Q1. CAN 통신에서 비트 마스킹을 사용한 이유가 무엇인가요?	CAN 데이터 필드는 8바이트(64비트)로 고정되어 있습니다. 6개 신호를 각각 별도 메시지로 전송하면 버스 부하가 6배 증가하므로, 비트 마스킹과 시프트 연산으로 여러 신호를 하나의 프레임에 패키징하여 대역폭 효율을 극대화하였습니다. 예를 들어 LFT(10비트)와 HEM(19비트)은 byte0~byte3에 걸쳐 배치되며, byte1 하나에 두 신호의 비트가 공존합니다. 이를 정확히 분리하려면 AND 마스킹으로 불필요한 비트를 제거하고 SHIFT로 원하는 비트 위치를 맞추는 과정이 필수입니다.
Q2. 해킹 감지 로직에서 Rate 기반 방식을 선택한 이유와 한계는?	CAN 프로토콜은 메시지 인증 헤더가 없어 데이터 내용만으로 정상/위조를 판별하기 어렵습니다. 반면 메시지 주입 공격은 반드시 비정상적인 빈도로 패킷을 전송하기 때문에, 도착 시간 간격(Rate)을

예상 질문	모범 답변 방향
	모니터링하는 방식이 구현 간단성과 실시간성 면에서 효과적입니다. 다만 동일 Rate 를 유지하며 데이터만 조작하는 Spoofing 은 탐지하지 못한다는 한계가 있으며, 개선책으로는 신호 값의 정상 범위 검사를 추가하는 Rule-based 방식이나 과거 패턴과 비교하는 통계적 이상 탐지 방법을 고려할 수 있습니다.
Q3. 응용 기능에서 음수 처리를 위해 오프셋 인코딩을 선택한 근거는?	CAN 데이터 필드의 비트는 부호 없는 정수로 패킹되므로, 음수를 직접 표현할 수 없습니다. 2의 보수 방식은 비트 폭 전체를 할당해야 하는데 STRESS 필드가 10 비트로 제한되어 있어, 간단한 오프셋 인코딩(+100 가산 후 패킹, 복원 시 -100 차감)을 채택했습니다. 이 방식은 STRESS 범위를 -100~+923 까지 표현할 수 있으며, 송수신 양단에서 동일한 오프셋 값을 공유하면 별도의 비트 오버헤드 없이 음수를 처리할 수 있습니다.
Q4. 팀장으로서 갈등 상황이나 어려움이 있었다면?	[추정] 적용 가능한 상황: 비트 맵 설계 초기에 각 신호의 필요 비트 수를 협의하는 과정에서, 특정 신호의 해상도를 높이면 다른 신호의 비트 폭이 줄어드는 트레이드오프 문제가 발생했습니다. 이를 해결하기 위해 각 신호의 의료적 의미와 실제 측정 정밀도 요구사항을 기준으로 우선순위를 설정하고, 팀원들과 공유하여 합의를 이끌어냈습니다. 이 경험을 통해 기술적 결정은 데이터만큼 팀의 공감각이 중요하다는 것을 배웠습니다.
Q5. LabVIEW 를 선택한 이유와, 다른 환경과의 차이점은?	LabVIEW 는 데이터 플로우(Data Flow) 패러다임을 기반으로, UI(프런트 패널)와 로직(블록 다이어그램)을 동시에 시각적으로 설계할 수 있습니다. 계측·제어 시스템 분야에서 산업 표준으로 활용되며, 파형 차트와 게이지 등 시각화 위젯이 내장되어 있어 실시간 모니터링 구현이 빠릅니다. C/Python 에 비해 하드웨어 I/O 연결과 신호 처리 파이프라인 구성이 직관적이라는 장점이 있습니다. 다만 Formula Node 를 제외한 일반 로직은 그래픽 배선 방식이라 복잡한 알고리즘에서 가독성이 떨어지는 단점도 있습니다.

9. 보고서 정확도 향상을 위한 추가 질문

본 보고서는 제공된 발표 슬라이드와 LabVIEW 캡처 화면만을 기반으로 작성되었습니다. 아래 정보를 추가로 제공해 주시면 보고서의 정확도를 높일 수 있습니다:

- Q1. 팀원 6 명의 실제 역할 분담이 기억나시나요? (재영, 준현, 성윤, 유정, 도현 각각의 담당 파트)
- Q2. 본인(채연)이 직접 구현한 LabVIEW 로직의 범위가 어디까지인지 확인 가능한가요? (Tx 만? Rx 도? 해킹 감지도?)
- Q3. 프로젝트 진행 중 가장 어려웠던 기술적 문제나 에피소드가 있으신가요? (자소서/면접 스토리텔링에 활용 가능)
- Q4. 응용 기능의 신호 실수 범위 설정 근거가 있나요? (예: 실제 의료 정상 참고치를 참조했는지?)
- Q5. 우수상 수상 시 심사위원의 피드백이나 특별히 언급된 강점이 있었나요?
- Q6. 발표 날짜가 2024 년 12 월 중 언제였는지, 그리고 수업명/교수명이 기억나신다면 기재 가능한가요?

본 보고서는 실제 프로젝트 자료를 기반으로 작성된 복원 문서입니다. [추정] 태그가 없는 내용은 발표 슬라이드 및 LabVIEW 캡처 화면에서 직접 확인된 사실 기반 정보입니다.